RedString Founding Engineer - What to Learn

MUST Learn (Class A keywords with no direct project experience)

- **FastAPI, SQLAlchemy & Alembic (Python backend)** – your backend production experience is Node.js/Express/tRPC, not Python. FastAPI is the closest bridge: async route handlers and Pydantic request validation map almost 1:1 onto Express middleware and Zod schemas you already use daily. SQLAlchemy is Python's ORM (similar role to Prisma); Alembic handles schema migrations (similar to Prisma Migrate). Build one small FastAPI + SQLAlchemy + Alembic CRUD service to have a concrete Python backend project.
- **LangGraph & deepagents** – LangGraph is LangChain's framework for building stateful, graph-based agent workflows (nodes = steps, edges = transitions, with cycles/branching) – a more structured version of the agent loops you've built manually (Vibe's Inngest AgentKit loop, Echo's tool-calling agent). deepagents is a newer, less common framework for the same space – worth a quick read of its docs since it's less documented elsewhere. Rebuild one of your existing agent loops (e.g. Echo's 3-tool support agent) in LangGraph to see the parallel directly.
- **Celery + RabbitMQ (Python async workers)** – distributed task queue with RabbitMQ as the message broker. You understand the underlying pattern well (Inngest's durable background jobs in Vibe), but Celery/RabbitMQ requires you to run and configure the queue infrastructure yourself rather than using a managed service. This is likely to come up given the JD's "asynchronous processing pipelines with queuing and failure recovery" responsibility.
- **Testing discipline: Vitest, Playwright, pytest, Hypothesis** – none of your current projects have test suites (audit-confirmed), and this JD explicitly wants "rigorous testing discipline and TDD experience." This is the single biggest practical gap to close: write tests for one existing project (Vitest for a frontend component, pytest for your FastAPI practice project) before interviewing.

GOOD to Learn (strengthens the application, not blocking)

- **MUI, Radix UI, Remotion** – you know Tailwind CSS and Shadcn UI (built on Radix primitives) already, so Radix UI itself is a small step; MUI is a different, more opinionated component library worth a skim. Remotion (programmatic video creation in React) is genuinely novel – relevant if RedString's product involves video/media pipelines (listed as a nice-to-have).
- **Redis Streams** – a specific Redis data structure for event streaming/log-based messaging, distinct from Redis's more common cache use. Worth 30 minutes of docs reading given it's explicitly named.
- **Hexagonal architecture** – a specific architectural pattern (ports and adapters) for isolating business logic from infrastructure. Conceptually close to Clean Architecture and to the layered thinking you already apply in Echo's monorepo design – mostly new vocabulary for an instinct you already have.

Fast-Ramp Note

This is one of the strongest AI/full-stack matches you've tailored for – your LangChain-adjacent LLM tool-calling and agent-loop experience (Echo, Vibe) maps directly onto this JD's core ask, and your Next.js/React/TypeScript frontend is a precise match. The two real gaps are a Python backend (FastAPI/SQLAlchemy) project to point to, and an actual test suite somewhere in your portfolio – both are worth building before interviewing, since "proven full-stack delivery with... Python backends" and "rigorous testing discipline" are both explicit must-haves.